

Understanding of Node js

Week 3/S1/Lecture #:BEE – 5

Topics Covered

- What is Node.js?
 - Why did we need Node.js? (The big problem before 2009)
 - What problems did developers face before Node.js?
 - How was Node.js created?
 - What are JavaScript Engines?
 - What is the V8 Engine?
 - How JavaScript ran in browsers BEFORE Node.js vs AFTER Node.js
 - Node.js Architecture – Full breakdown (super simple)
 - Call Stack
 - Event Loop
 - Blocking vs Non-Blocking
 - libuv, Thread Pool, and how heavy work is handled
 - Real-world industry examples explained like stories
 - Key Takeaways
-

1. What is Node.js ?

Node.js is a free, open-source, runtime environment that allows developers to run JavaScript code outside the browser—directly on the server/computer using Google's V8 JavaScript engine.

- **Simple meaning :**
Imagine JavaScript was a kid who was only allowed to play inside the browser playground (Chrome, Firefox).
In 2009, Ryan Dahl said, “Let this kid come out and play on the big ground (server) also!”
So he built a new playground called Node.js where JavaScript can now do server work like talking to databases, handling files, creating APIs, etc. 2015: ES6 (ES2015) revolutionized JavaScript with classes, arrow functions, promises

Real-life example:

Netflix, Uber, PayPal, Walmart – all use Node.js on their servers because one language (JavaScript) is used both on frontend and backend → less confusion, faster development.

2. Why Did We Need Node.js? (The Real Pain Before 2009)

Before Node.js, if you wanted to build a website/chat app, you had to use different languages:

Frontend → JavaScript

Backend → PHP, Java, Ruby, Python, etc.

Problem: You need to learn and manage 2 completely different languages and teams.

Worse problem: Traditional servers (Apache, Nginx with PHP/Java) were slow with thousands of users waiting at the same time (like Zomato during dinner time).

They used “one thread per user” model → if 10,000 users come, server creates 10,000 threads → RAM finishes → server dies.

Node.js came and said: “I will handle 10,000 users with just ONE thread using event-loop magic.”

3. What Problems Did Developers Face Before Node.js?

- Blocking I/O → Everything waits
Reading a file or database query would freeze the entire server until it finishes.
- Thread explosion → Server crashes with too many users
Each user got one thread → 10,000 users = 10,000 threads = dead server.
- Different languages → Mental overhead
Frontend team (JS) and backend team (Java/Python) couldn't understand each other's code easily.
- Callbacks hell in early solutions → Code becomes unreadable.

Real example:

Imagine BigBasket server in 2008 – one user uploads photo → server freezes for everyone else until photo saves → other users see loading... loading... → they leave.

4. How Was Node.js Created?

2009 – Ryan Dahl (genius developer) was frustrated.

He wanted:

- JavaScript on server
- Super fast with many users
- Non-blocking everything

He took:

Google's V8 Engine (super fast JS engine from Chrome)

- libuv (C library made by Joyent for async I/O)
- His own event loop magic

→ Mixed them → Node.js was born in 2009.

First name was "web.js" → later became Node.js.

5. What are JavaScript Engines?

- Engine = the machine that converts your JavaScript code into something computer understands (machine code).
- Popular engines:
 - V8 → Chrome & Node.js
 - SpiderMonkey → Firefox
 - JavaScriptCore → Safari
 - Chakra → Old Edge
- They all do the same job: take JS → compile fast → run fast.

6. What is V8 Engine? (The Heart of Node.js)

V8 is Google's super-fast JavaScript engine written in C++.

It does two magic things:

1. JIT Compilation (Just-In-Time)
→ Converts JS → machine code while running (not before, not after – exactly when needed)
2. Ignition + TurboFan → makes JS run almost as fast as C++ code.

Real-life example:

You write `console.log("hello");`

V8 converts it to machine code in microseconds and runs it directly on CPU – no slow interpreter.

7. How JavaScript Ran in Browser BEFORE Node.js vs AFTER Node.js

BEFORE Node.js (Only in Browser):

JavaScript could only run inside browser → limited to DOM, alerts, AJAX.

Could not read/write files, could not open servers, could not access system.

AFTER Node.js:

Same V8 engine came out of browser → now JavaScript can:

- Read/write files
- Create web servers
- Connect to databases
- Run on Linux/Windows servers 24×7

Real-life example:

Before: Instagram website could show photos (frontend JS)

After Node.js: Instagram can now receive your uploaded photo on server using JavaScript only (no PHP/Java needed).

8. Node.js Architecture – Explained Like a Kitchen in a Busy Restaurant

Imagine Node.js is a very smart single chef (single thread) in a restaurant who never waits.

Components:

- **Call Stack (The Chef's Hands)**
 - Only one thing at a time.
 - LIFO (Last In First Out).
 - Example: You order momos → chef starts boiling → while waiting, he doesn't stand idle → he takes next order.
- **Event Loop (The Smart Waiter)**
 - Keeps checking: "Is anything ready to serve?"
 - Runs in infinite loop.
 - Super lightweight – can handle 100,000 users easily.
- **Blocking vs Non-Blocking**
- **Blocking (Bad – traditional way):**
 - Chef starts boiling momos → stands and waits 10 minutes → no new orders taken → customers angry.
- **Non-Blocking (Node.js way):**
 - Chef puts momos in steamer → gives timer → immediately takes next order → when timer rings, he serves.
- **libuv Library (The Kitchen Helpers)**
- libuv is the secret magic library (written in C) that handles all heavy work.
- It gives Node.js:
 - File system access
 - DNS lookup
 - Network I/O
 - Thread Pool (4 helpers by default)
- **Thread Pool (4 Strong Helpers in Kitchen)**
- When something is really heavy (like reading a 5GB file), even non-blocking is slow on main thread.

So libuv says: "Main chef, you continue taking orders. I'll give this heavy job to one of my 4 helpers (threads)."

Default: 4 threads

You can increase to 100 if needed (`process.env.UV_THREADPOOL_SIZE = 100`)

Real-life example :

PhonePe during New Year sale – 1 lakh payments per second.

All database writes go to thread pool → main event loop never waits → app never slows down.

9. Real-World Industry Examples

- **Netflix**
 - 200 million users streaming
 - Node.js handles millions of “What to watch next?” requests without blocking
 - One slow recommendation = everyone waits (not acceptable)
- **Uber**
 - Driver location updates every 4 seconds
 - Millions of WebSocket connections open
 - Node.js event loop handles all without creating millions of threads
- **PayPal**
 - Switched from Java to Node.js
 - Applications became 2x faster
 - Lines of code reduced by 33%
 - Same team can work on frontend + backend

10. Key Takeaways

- Node.js = JavaScript running on server using V8 + libuv
- Single thread + Event Loop + Non-blocking = Magic for handling thousands of users
- Before Node.js → servers died with traffic
- After Node.js → one cheap server can handle what 50 old servers used to do
- Event Loop never sleeps, never waits, never blocks
- Heavy work → thread pool (4 threads)
- Light work → event loop handles directly
- You write simple JavaScript → Node.js makes it super fast and scalable



Practice Activities

1. Explain Node.js in one line as you would tell your TL in interview.
(Best answer gets you selected instantly)
2. JavaScript was originally made for browsers only. Then how can it read files and create servers in Node.js?
3. What is the exact meaning of "Node.js is single-threaded"?
Does it mean Node.js can do only one thing at a time? Explain with example.
4. Your Node.js server is handling 50,000 users right now.
How many threads are running at this moment? Explain clearly.
5. What is the difference between Blocking and Non-Blocking I/O?
Give a real restaurant example (not copy-paste).
6. What is libuv? Who created it and why?
Without libuv, can Node.js still work?
7. By default how many threads are there in the thread pool?
How do you increase it in production? Write the one-line code.
8. What are the 4 major things libuv handles for Node.js?
9. Event Loop has 6 phases. Name any 4 phases and tell which phase handles setTimeout and which handles fs callbacks.
10. Ryan Dahl now says he has 6 regrets about Node.js. Name at least 3 regrets he has.
11. Why did Netflix, Uber, PayPal switch to Node.js? Give exact technical reason (not "it's fast").
12. Traditional Java/PHP servers use one thread per request model.
Why does that model fail during flash sales (like Flipkart Big Billion Day)?
13. You are building a video streaming backend like Hotstar.
14. 1 million users are watching IPL final together.
15. Explain how Node.js will handle it without crashing.